

VentureMiner AI

Technical Requirements Document (TRD)

Document: 02-TRD / 02_technical_requirements_document

Version: v1.0

Date: 2026-07-20

Author: VentureMiner AI Documentation Team

Status: Approved

Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Technical Requirements Document (TRD)

Table of Contents

1. Purpose & Scope

1.1 Audience

1.2 Out of scope

2. Technology Stack

2.1 High-level summary

2.2 Rationale (selected)

3. System Topology

3.1 Regions

3.2 High-level diagram

3.3 Network

4. Service Decomposition

4.1 Service catalog

4.2 Communication

4.3 Service principles

5. Data Architecture

5.1 Primary store

5.2 Schemas (high-level)

5.3 Vector storage

5.4 Object storage

5.5 Caching

5.6 Search

6. AI & LLM Architecture (overview)

7. Integration Architecture

8. Performance Engineering

8.1 Targets (NFRs from PRD §8.1)

8.2 Techniques

8.3 Performance budgets (per page)

9. Scalability & Capacity Planning

9.1 Scaling axes

9.2 Targets

- 11. Observability
 - 11.1 Pillars
 - 11.2 SLOs
 - 11.3 Alerting
- 12. Disaster Recovery & Business Continuity
- 13. Compliance
- 14. Internationalization & Localization
- 15. Developer Experience
- 16. Open Standards & Interop
- 17. Risks & Open Questions
- 18. Appendix
 - 18.1 Revision history
 - 18.2 Cross-references

Technical Requirements Document (TRD)

Document 02 — VentureMiner AI The canonical technical specification. This document translates the PRD's what into the how at the platform, infrastructure, and architecture level. It is the source of truth for the stack, deployment topology, performance, and non-functional guarantees.

Table of Contents

1. Purpose & Scope
2. Technology Stack
3. System Topology
4. Service Decomposition
5. Data Architecture
6. AI & LLM Architecture (overview)
7. Integration Architecture
8. Performance Engineering
9. Scalability & Capacity Planning
10. Security Architecture
11. Observability
12. Disaster Recovery & Business Continuity
13. Compliance
14. Internationalization & Localization
15. Developer Experience
16. Open Standards & Interop
17. Risks & Open Questions
18. Appendix

1. Purpose & Scope

This document is the technical contract for the VentureMiner AI platform. Every architectural decision traces back to a requirement in the PRD (Document 01) and an NFR in Section 8 of that document. Anything not in this TRD is not part of v1.0 scope.

1.1 Audience

- Backend, frontend, AI, and platform engineers.
- SRE / DevOps.
- Security and compliance reviewers.
- External integrators (via the public API).

1.2 Out of scope

- Pricing strategy (Document 01 §12).
- Marketing surface (handled by Web team outside this suite).

2. Technology Stack

2.1 High-level summary

Frontend web	Next.js 15 (App Router) + React 19 + TypeScript	SSR for SEO, mature ecosystem
Design system	Tailwind CSS + Radix UI + custom kit	Speed + a11y baseline
API gateway	Kong (managed)	Mature, plugin-rich, supports OIDC
Backend services	Python 3.12 (FastAPI) + Node.js 22 (TypeScript)	ML/Python ecosystem, async
AI orchestration	Custom multi-agent runtime on LangGraph + MCP	Full control, RAG-friendly
Primary DB	PostgreSQL 16 (managed, HA)	OLTP, JSONB, vector extension
Vector DB	pgvector (v1) → Qdrant (v2)	Single stack v1; scale v2
Cache	Redis 7 (managed)	Sessions, rate limits, hot cache
Search	OpenSearch 2.x (managed)	Full-text + facets
Object store	S3-compatible (AWS S3 → Cloudflare R2)	Reports, exports
Queue / broker	NATS JetStream	Lightweight, durable streams
Background jobs	Temporal	Durable, retryable workflows
Identity	Auth0 (managed) for v1; self-host option v2	SOC 2-ready, fast start
Billing	Stripe	Industry standard
Email	Postmark	Deliverability
LLM provider	Anthropic Claude (primary), OpenAI (fallback), self-host (v2)	Quality, availability
Observability	OpenTelemetry → Datadog (managed)	Unified traces/logs/metrics
CI/CD	GitHub Actions + Argo CD	GitOps
IaC	Terraform	Multi-cloud ready
Hosting	AWS (primary), GCP (DR)	Multi-region

2.2 Rationale (selected)

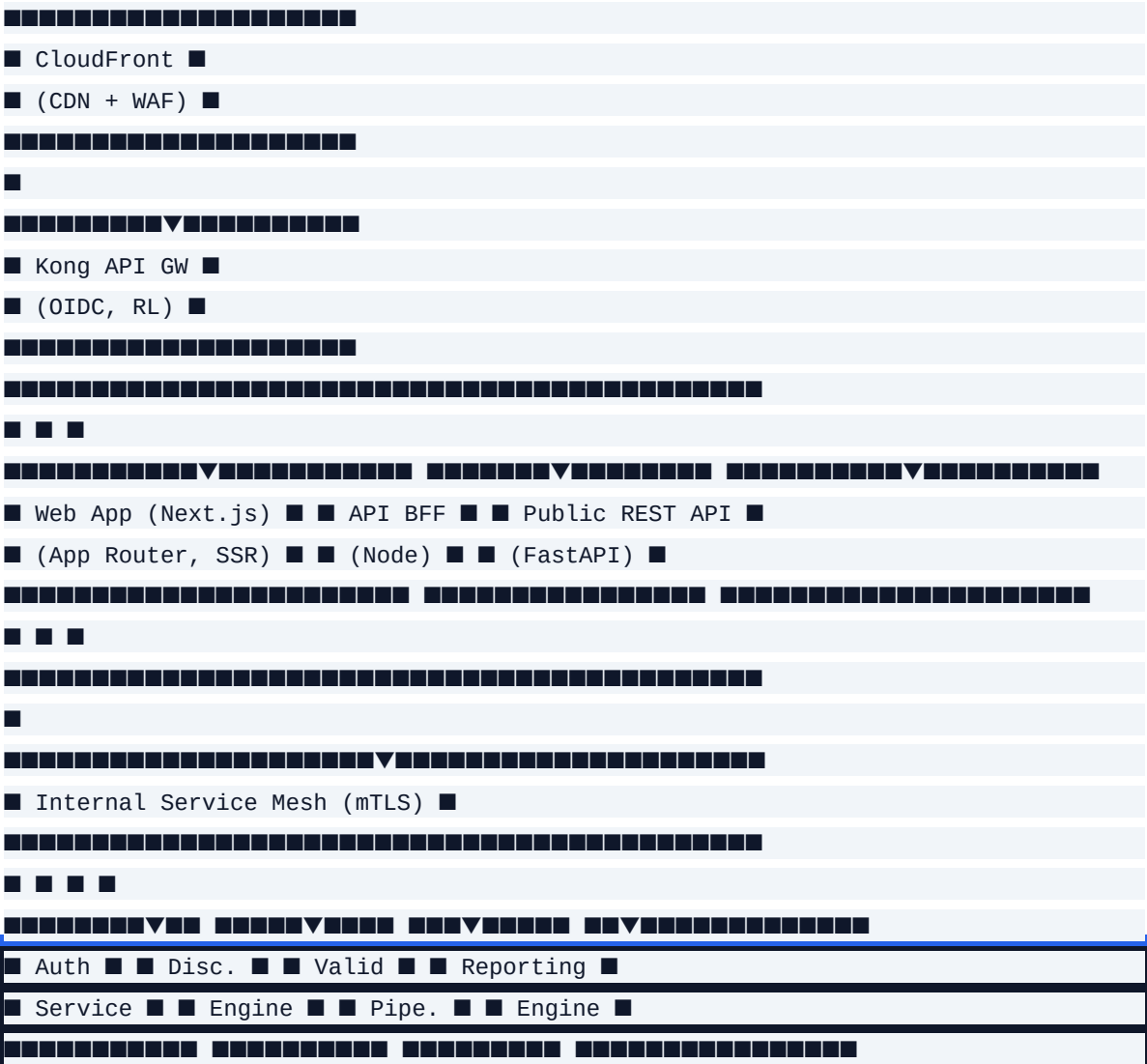
- Python on FastAPI for AI-heavy services — best-in-class ML/LLM libraries (LangChain, pydantic, numpy, pandas).
- Node/TypeScript for the user-facing API and webhooks — strong typing, ecosystem fit.
- pgvector for v1 vector search — avoids operational sprawl; we know the limit (~10M vectors/well-tuned instance) and stay under it.
- Temporal for workflows — durable retries, versioning, visibility out of the box.
- Auth0 — shortens time-to-SOC-2 vs building our own.

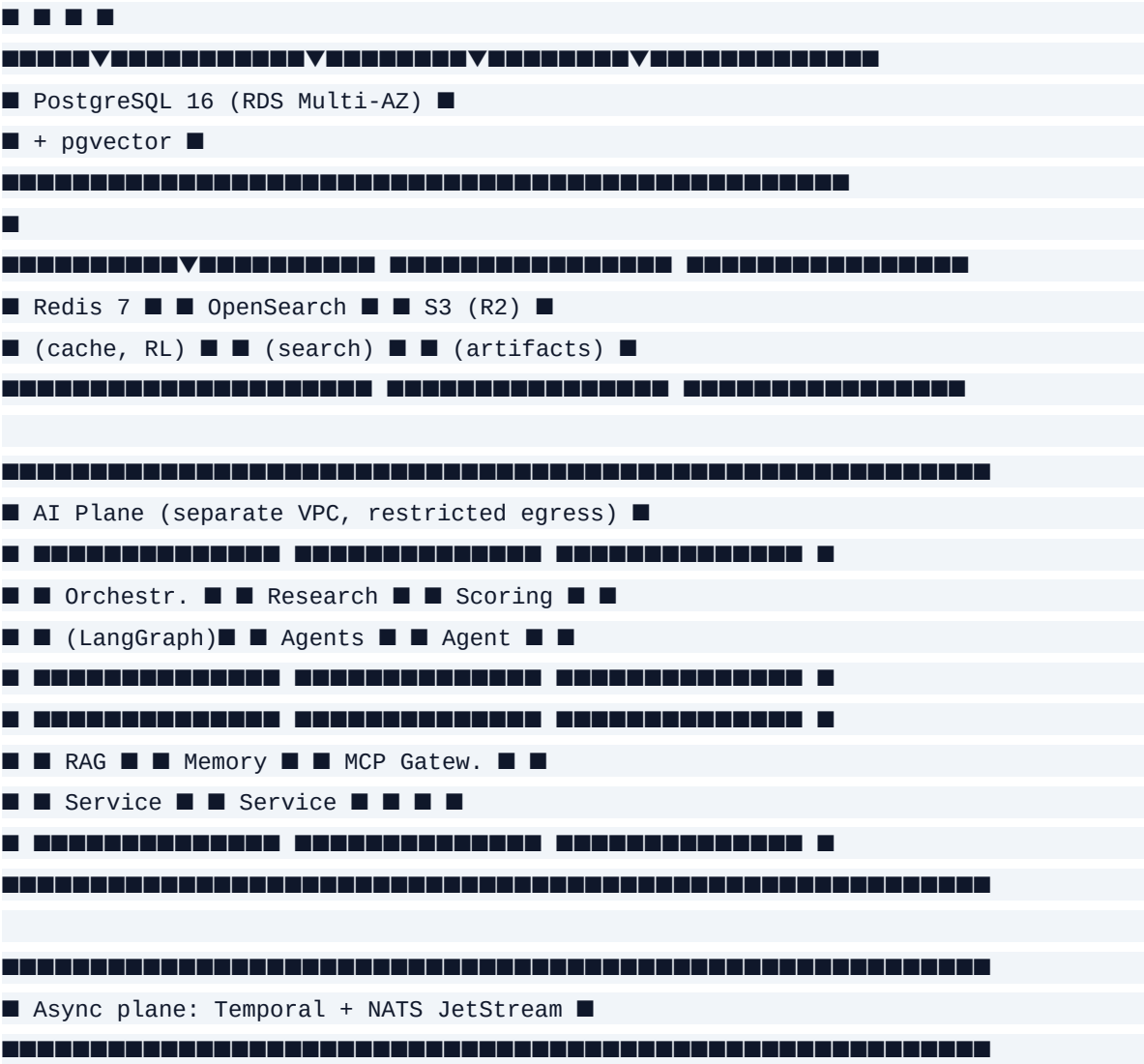
3. System Topology

3.1 Regions

- Primary: us-east-1 (Virginia).
- Secondary (read + DR): us-west-2 (Oregon).
- EU (configurable residency, post-MVP): eu-west-1 (Dublin).
- APAC (post-MVP): ap-northeast-1 (Tokyo).

3.2 High-level diagram





3.3 Network

- All internal traffic is mTLS.
- Egress to LLM providers is via NAT gateway with allow-listed IPs.
- AI plane runs in a separate VPC with no direct inbound from the public internet.

4. Service Decomposition

4.1 Service catalog

web	TypeScript / Next.js	UI, SSR, BFF for browser
api-public	Python / FastAPI	Public REST API, OpenAPI spec
api-bff	TypeScript / Node	Internal BFF for the web app
auth-svc	Python	Sessions, RBAC, SSO callbacks

web	TypeScript / Next.js	UI, SSR, BFF for browser
workspace-svc	Python	Workspaces, members, projects
opportunity-svc	Python	Opportunity CRUD, lifecycle
discovery-svc	Python	Orchestrates discovery runs
validation-pipeline	Python	Orchestrates validation runs
scoring-svc	Python	Rubric, scoring, history
reporting-svc	Python	Report assembly, exports
agent-runtime	Python	Hosts LangGraph agents, MCP gateway
rag-svc	Python	RAG retrieval, indexing
memory-svc	Python	Long-term memory store
plugin-svc	Python	Tool registry, plugin runtime
source-svc	Python	Source connectors (HTTP, RSS, APIs)
search-svc	Python	OpenSearch index management
billing-svc	TypeScript	Stripe webhooks, entitlements
notify-svc	TypeScript	Email, Slack, webhooks
audit-svc	Python	Append-only audit log

4.2 Communication

- Synchronous: gRPC for service-to-service, REST for external.
- Asynchronous: NATS JetStream for fan-out, Temporal for durable workflows.
- Events: CloudEvents v1.0 schema.

4.3 Service principles

- Each service has its own data store (no shared DBs).
- Each service is independently deployable.
- Each service exposes a versioned OpenAPI or Protobuf contract.
- Each service emits OpenTelemetry traces.

5. Data Architecture

5.1 Primary store

- One logical database per service (physical DBs in the same cluster for v1, separate clusters per service in v2).

5.2 Schemas (high-level)

auth	auth-svc	user, account, session, oauth_link, mfa, audit_event
workspace	workspace-svc	workspace, membership, project, invite
opportunity	opportunity-svc	opportunity, opportunity_tag, opportunity_link, opportunity_event
discovery	discovery-svc	discovery_run, discovery_source, discovery_hit
validation	validation-pipeline	validation_run, validation_step, validation_evidence
scoring	scoring-svc	rubric, rubric_version, score, score_breakdown
reporting	reporting-svc	report, report_section, report_export
agent	agent-runtime	agent_run, agent_step, agent_tool_call
rag	rag-svc	document, chunk, embedding
memory	memory-svc	memory_record, memory_link
plugin	plugin-svc	plugin, plugin_version, plugin_installation
billing	billing-svc	subscription, invoice, entitlement
notify	notify-svc	notification, notification_pref
audit	audit-svc	audit_event (append-only)

5.3 Vector storage

- v1: pgvector in rag schema.
- v2: migrate to Qdrant when embedding count exceeds 5M.

5.4 Object storage

- Reports, exports, raw corpus documents: S3-compatible object store.

- Lifecycle: hot (0–30d) → warm (30–180d) → cold (180d+).

5.5 Caching

- Redis 7 for sessions, rate limits, hot opportunities, per-workspace feature flags.
- TTL policies: sessions (8h idle / 30d max), feature flags (5m), opportunities (24h).

5.6 Search

- OpenSearch for full-text and faceted search over opportunities, reports, and source documents.
- Reindexed asynchronously on writes (write-through cache, debounced bulk indexer).

6. AI & LLM Architecture (overview)

The AI plane is detailed in Documents 07–18. Summary:

- Multi-agent runtime built on LangGraph.
- Model Context Protocol (MCP) exposes tools to agents via a uniform interface.
- RAG is the only path to external knowledge for non-parametric claims; every claim carries a citation.
- Memory distinguishes session (Redis), user (PostgreSQL), and platform (Postgres + vector) memory.
- Plugins are first-class artifacts with versioned manifests and policy-based sandboxing.
- LLM provider is Anthropic Claude (Sonnet for routine, Opus for high-stakes synthesis), with OpenAI as a fallback.

7. Integration Architecture

- Public REST API — OpenAPI 3.1, OAuth 2.0, scoped tokens, versioned URL (/v1/).
- Webhooks — outbound, signed (HMAC-SHA256), retry with exponential backoff, dead-letter queue.
- Native integrations — Slack (incoming webhooks + Bolt), Notion (push), Stripe (webhooks).
- Source connectors — abstracted via the source-svc (Document 10 — Plugin Architecture).
- MCP gateway — exposes internal tools to external MCP-aware clients (v2).

8. Performance Engineering

8.1 Targets (NFRs from PRD §8.1)

Dashboard TTI	< 1.5s p75
One-page brief	< 60s p75
Full report (Standard)	< 8 min p75
Search	< 300ms p75
API p95	< 500ms (sync)

8.2 Techniques

- Edge cache for static assets (1y) and HTML (5m, stale-while-revalidate).
- DB read replicas for dashboard reads.
- Materialized views for portfolio and trends.
- Streaming responses for LLM-generated content (SSE / WebSocket).
- Pre-warm RAG indexes per top workspace.
- Inference routing — fast model for routine synthesis, deep model for long-form reports.

8.3 Performance budgets (per page)

Landing	80 KB	15 KB	0	1.0s
Dashboard	180 KB	20 KB	4	1.5s
Report	220 KB	25 KB	6	2.0s

9. Scalability & Capacity Planning

9.1 Scaling axes

- Stateless API services — horizontal autoscale on CPU + RPS.
- Stateful services (Postgres, OpenSearch, Redis) — vertical scale + read replicas.
- AI runtime — horizontal autoscale on queue depth.
- Background jobs — Temporal scales workers horizontally.

9.2 Targets

Concurrent users	2,000	10,000	50,000
Opportunities	5M	25M	100M
Documents in RAG	20M	100M	500M
Reports / day	5,000	25,000	100,000
LLM tokens / day	200M	1.5B	6B

9.3 Cost ceilings (per 1k reports)

- LLM tokens: ≤ \$4.00
- Compute: ≤ \$1.20
- Storage: ≤ \$0.10
- Total CoGS target: ≤ \$5.50 (≤ 30% of revenue at Team plan).

10. Security Architecture

- AuthZ: RBAC at workspace level + per-resource ACLs.
- Encryption: TLS 1.3 in transit, AES-256 at rest, per-tenant DEKs in KMS.
- Secrets: AWS Secrets Manager; no secrets in env or code.
- Network: VPC isolation, WAF, private subnets for stateful services.
- Application security: SAST (Snyk), DAST (OWASP ZAP), dependency scanning (Dependabot).
- Tenant isolation: row-level security in Postgres, namespace in object store.
- PII: pseudonymized in logs, redacted in LLM prompts.
- Threat model: documented in Document 21, reviewed every 90 days.

11. Observability

11.1 Pillars

- Logs — structured JSON, shipped to Datadog, 30-day hot, 1y cold.
- Metrics — Prometheus-style, 13-month retention.
- Traces — OpenTelemetry, 100% sampling for AI pipeline, 10% for API.
- RUM — Datadog RUM for web, with PII redaction.
- Synthetics — 5-min uptime, 30-min API smoke.

11.2 SLOs

Public API	success rate	99.9% (30d)
Dashboard	TTI < 1.5s p75	99.5%
Report generation	completes in < 10 min p95	99%
Discovery	fresh (last 24h) opportunity returned in < 30s p95	99%

11.3 Alerting

- Page on SLO burn rate $\geq 2x$ in 1h.
- Ticket on SLO burn rate $\geq 1x$ in 24h.
- Weekly error budget review.

12. Disaster Recovery & Business Continuity

RPO	1 hour
RTO	4 hours
Backups	Daily DB, hourly WAL streamed to S3
DR region	us-west-2 (warm standby)
DR drill	Quarterly

- Static assets and AI artifacts replicated cross-region.

13. Compliance

SOC 2 Type II	GA	Security + Compliance
ISO 27001	Month 18	Security + Compliance
GDPR	Launch	Legal + Security
CCPA	Launch	Legal + Security
HIPAA	Not in scope v1	—
EU AI Act	High-risk classification TBD; mitigations in place	Legal + AI

14. Internationalization & Localization

- All strings externalized.
- Date/time/number formatting via Intl.
- Currency formatting respects user preference.
- Right-to-left support deferred to v3.
- Locales at launch: en-US.
- v2 locales: es-ES, de-DE, ja-JP.

15. Developer Experience

- Monorepo with pnpm workspaces + Turborepo.
- Service templates via a Yeoman generator (vma scaffold).
- Local stack via Docker Compose + a one-command dev up.
- Database migrations via Atlas (declarative).
- Contract testing with Pact for cross-service consumers.
- Feature flags via LaunchDarkly (managed).
- Pre-commit hooks for lint, format, and security scans.

16. Open Standards & Interop

- OpenAPI 3.1 for REST APIs.
- AsyncAPI 2.6 for event schemas.
- CloudEvents 1.0 for event envelopes.
- JSON Schema 2020-12 for internal data contracts.

• OAuth 2.1 + OIDC for identity.

- SCIM 2.0 for user provisioning.

17. Risks & Open Questions

TRD-RISK-001	LLM provider concentration	Mitigated via multi-provider	AI Lead
TRD-RISK-002	pgvector scale ceiling	Plan: migrate to Qdrant v2	Platform
TRD-RISK-003	Auth0 vendor lock-in	Open: design self-host fallback	Auth Lead
TRD-RISK-004	DR cost	Open: tune to budget	SRE
TRD-OPEN-005	EU AI Act risk classification	Open: legal review	Legal
TRD-OPEN-006	Data residency for Enterprise EU	Open: scope & timeline	Product
TRD-OPEN-007	Self-hosted LLM option	Defer to v2	AI Lead

18. Appendix

18.1 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

18.2 Cross-references

- PRD: Document 01.
- Backend & DB: Document 05.
- AI Architecture: Documents 07–18.
- Engineering: Documents 19–30.

End of Document 02 — TRD. This document is the canonical technical contract. Any deviation must update this document and trigger a MAJOR version bump per Document 00 §6.